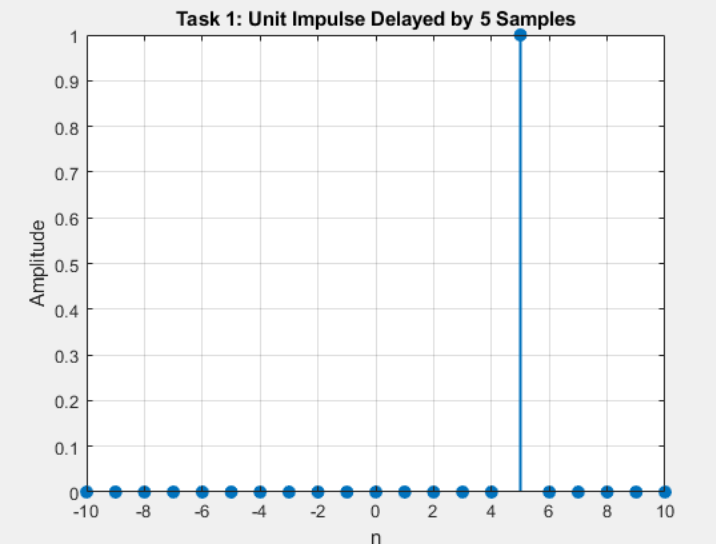
```
% Define discrete time vector
n = -10:10;

% Unit impulse is 1 at n = 0. To delay by 5, it is 1 at n = 5
impulse_delayed = (n == 5);

figure;
stem(n, impulse_delayed, 'filled', 'LineWidth', 1.5);
title('Task 1: Unit Impulse Delayed by 5 Samples');
xlabel('n'); ylabel('Amplitude');
grid on;
```

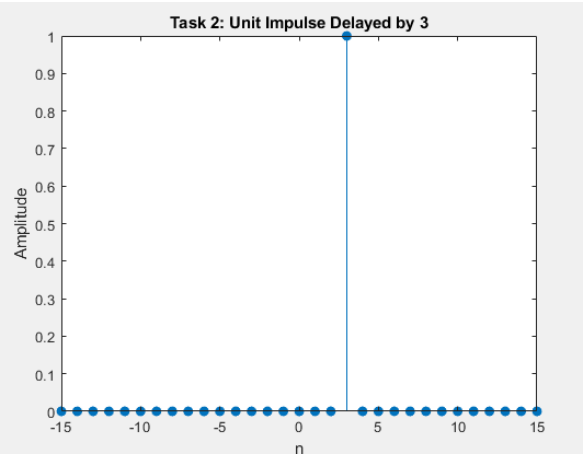


2

```
n = -15:15;
% User inputs
shift_val = input('Enter the number of samples to shift: ');
shift_dir = input('Shift right (delay) or left (advance)? (Enter "r" or "l"): ', 's');

% Determine shift logic
if strcmpi(shift_dir, 'r')
    impulse_shifted = (n - shift_val == 0); % Delay
    action = 'Delayed';
elseif strcmpi(shift_dir, 'l')
    impulse_shifted = (n + shift_val == 0); % Advance
    action = 'Advanced';
else
    error('Invalid input direction. Please use "r" or "l".');
end

figure;
stem(n, impulse_shifted, 'filled');
title(['Task 2: Unit Impulse ', action, ' by ', num2str(shift_val)]);
xlabel('n'); ylabel('Amplitude');
```

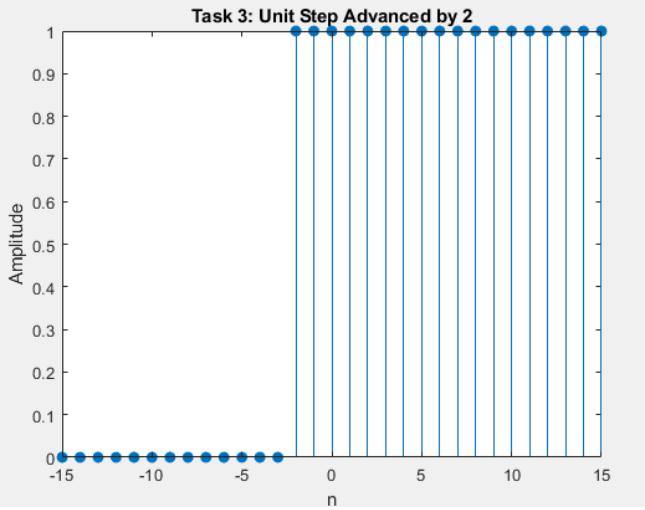


3

```
n = -15:15;
shift_val = input('Enter the number of samples to shift: ');
shift_dir = input('Shift right (delay) or left (advance)? (Enter "r" or "l"): ', 's');

if strcmpi(shift_dir, 'r')
    step_shifted = (n - shift_val >= 0); % Delay
    action = 'Delayed';
elseif strcmpi(shift_dir, 'l')
    step_shifted = (n + shift_val >= 0); % Advance
    action = 'Advanced';
else
    error('Invalid input direction.');
end

figure;
stem(n, step_shifted, 'filled');
title(['Task 3: Unit Step ', action, ' by ', num2str(shift_val)]);
xlabel('n'); ylabel('Amplitude');
```



4,5

```
function [x] = flipadv(u, s)
    % FLIPADV Flips the sequence first, then advances it by 's' samples.
    % u: input sequence array
    % s: number of samples to advance (shift left)

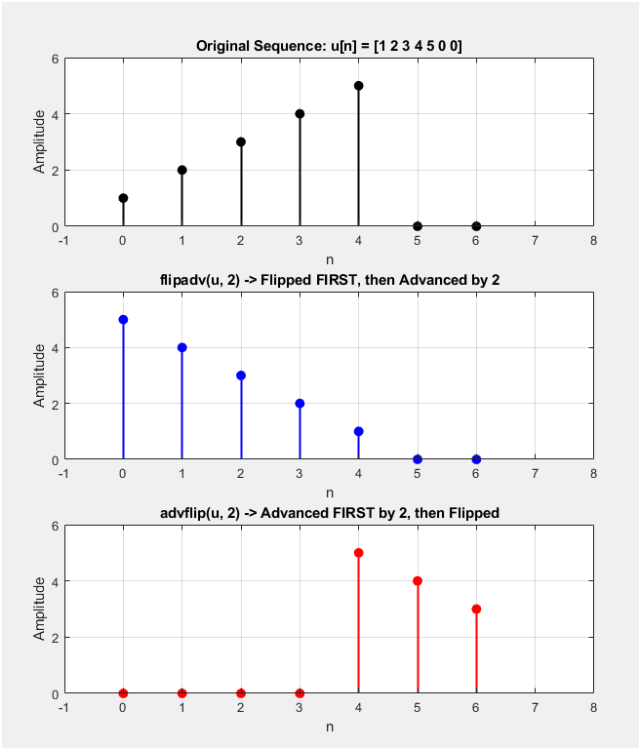
    u_flipped = fliplr(u); % Flip horizontally

    % Advancing a finite array means shifting left and padding zeros at the end
    if s > 0 && s < length(u)
        x = [u_flipped(s+1:end), zeros(1, s)];
    elseif s >= length(u)
        x = zeros(1, length(u)); % Fully shifted out
    else
        x = u_flipped;
    end
end
```

```
function [x] = advflip(u, s)
    % ADVFLIP Advances the sequence first by 's' samples, then flips it.
    % u: input sequence array
    % s: number of samples to advance

    % Advance first
    if s > 0 && s < length(u)
        u_advanced = [u(s+1:end), zeros(1, s)];
    elseif s >= length(u)
        u_advanced = zeros(1, length(u));
    else
        u_advanced = u;
    end

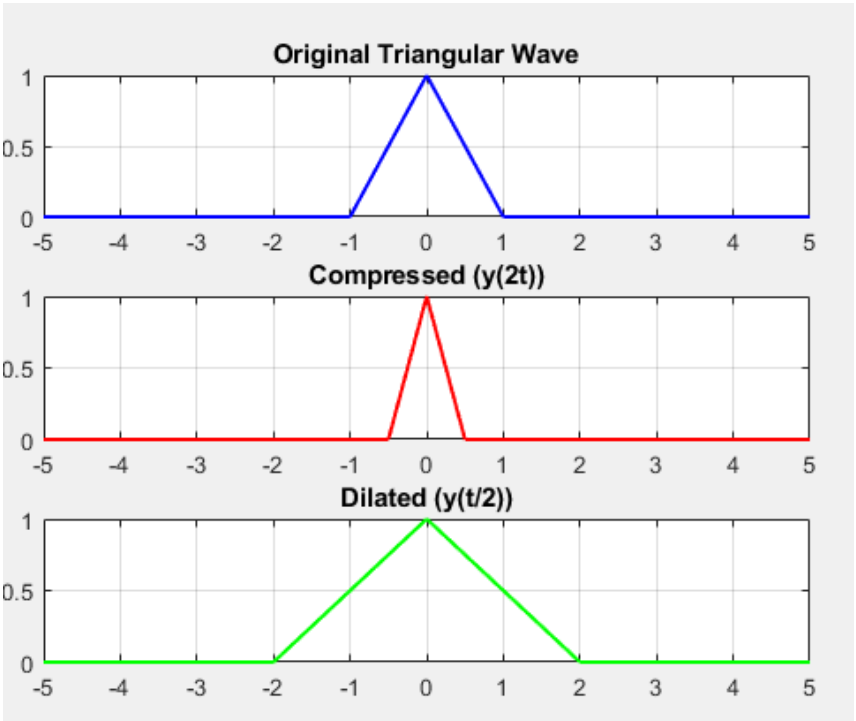
    % Then flip
    x = fliplr(u_advanced);
end
```



6

```
t = -5:0.01:5;
y_orig = tripuls(t, 2);      % Original (width 2)
y_comp = tripuls(2*t, 2);    % Compressed by factor of 2
y_dil  = tripuls(t/2, 2);    % Dilated by factor of 2

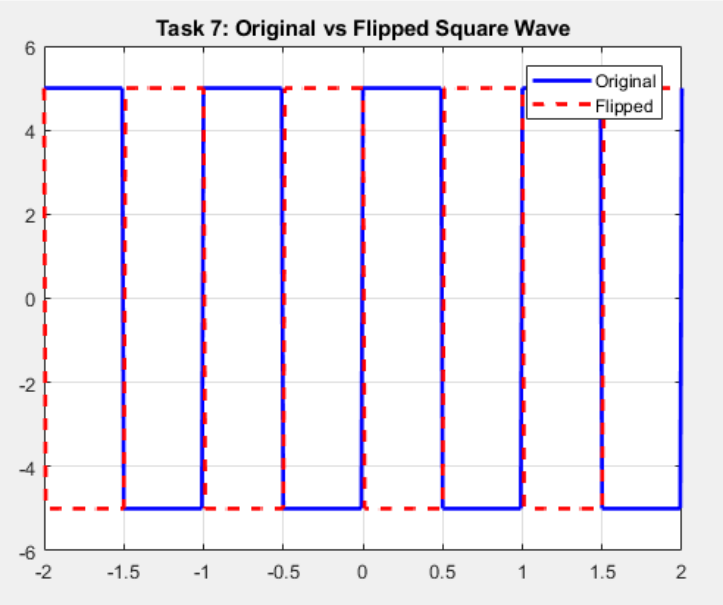
figure;
subplot(3,1,1); plot(t, y_orig, 'b', 'LineWidth', 1.5); title('Original Triangular Wave'); grid on;
subplot(3,1,2); plot(t, y_comp, 'r', 'LineWidth', 1.5); title('Compressed (y(2t))'); grid on;
subplot(3,1,3); plot(t, y_dil, 'g', 'LineWidth', 1.5); title('Dilated (y(t/2))'); grid on;
```



7

```
t = -2:0.01:2;
% Generate square wave. square(2*pi*t) oscillates between -1 and 1.
y_square = 5 * square(2*pi*t);
y_flipped = fliplr(y_square); % Flip the sequence

figure;
plot(t, y_square, 'b', 'LineWidth', 2); hold on;
plot(t, y_flipped, 'r--', 'LineWidth', 2);
title('Task 7: Original vs Flipped Square Wave');
legend('Original', 'Flipped');
axis([-2 2 -6 6]); grid on; hold off;
```



8

```
% visual_proof_linearity.m
n = -10:20;

% Define Inputs and Constants
x1 = (n - 1 >= 0);           % Unit Step (Step time = 1)
x2 = 3 * (n - 5 >= 0);       % Unit Step (Step time = 5, Final value = 3)
a1 = 7; a2 = 19;             % Constants

% Define Systems
sys1 = @(x) [zeros(1,3), x(1:end-3)];       % Sys1: y(n) = x(n-3)
sys2 = @(x) 5 .* [zeros(1,7), x(1:end-7)];  % Sys2: y(n) = 5*x(n-7)
sys3 = @(x) ([zeros(1,1), x(1:end-1)]).^2;  % Sys3: y(n) = x^2(n-1)

% Combined Input Sequence
x_combined = a1*x1 + a2*x2;

figure('Name', 'Linearity Proof', 'Position', [100 100 900 700]);
```

```
%% System 1: y(n) = x(n-3)
LHS1 = sys1(x_combined);
RHS1 = a1*sys1(x1) + a2*sys1(x2);

subplot(3,2,1); stem(n, LHS1, 'b', 'filled'); title('Sys1 LHS: H(a1x1 + a2x2)'); grid on;
subplot(3,2,2); stem(n, RHS1, 'r', 'filled'); title('Sys1 RHS: a1H(x1) + a2H(x2)'); grid on;

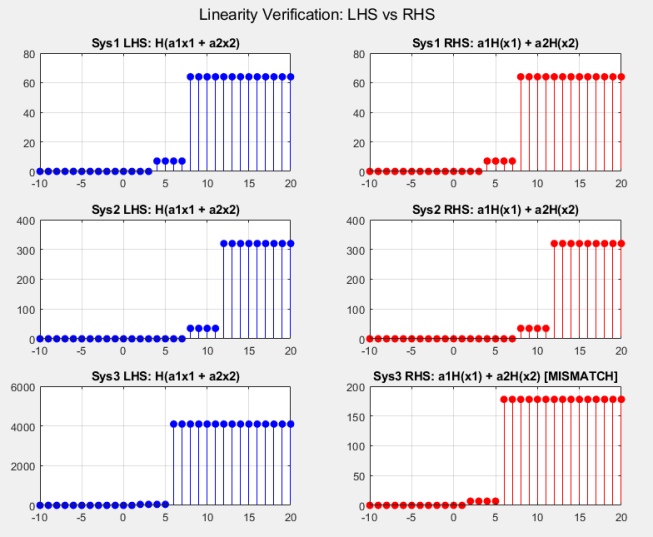
%% System 2: y(n) = 5*x(n-7)
LHS2 = sys2(x_combined);
RHS2 = a1*sys2(x1) + a2*sys2(x2);

subplot(3,2,3); stem(n, LHS2, 'b', 'filled'); title('Sys2 LHS: H(a1x1 + a2x2)'); grid on;
subplot(3,2,4); stem(n, RHS2, 'r', 'filled'); title('Sys2 RHS: a1H(x1) + a2H(x2)'); grid on;

%% System 3: y(n) = x^2(n-1)
LHS3 = sys3(x_combined);
RHS3 = a1*sys3(x1) + a2*sys3(x2);

subplot(3,2,5); stem(n, LHS3, 'b', 'filled'); title('Sys3 LHS: H(a1x1 + a2x2)'); grid on;
subplot(3,2,6); stem(n, RHS3, 'r', 'filled'); title('Sys3 RHS: a1H(x1) + a2H(x2) [MISMATCH]'); grid on;

suptitle('Linearity Verification: LHS vs RHS');
```



11,13

```
% Tasks 11 and 13: Command Window Verification of TI and LTI
% No plots - Outputs results directly to the Command Window

n = -10:20;
x = (n >= 0); % Standard unit step as test signal
delay_k = 4;  % Delay factor

% Define the Systems from the previous tasks
sys1 = @(x_seq) [zeros(1,3), x_seq(1:end-3)];       % System 1: y(n) = x(n-3)
sys2 = @(x_seq) 5 .* [zeros(1,7), x_seq(1:end-7)];  % System 2: y(n) = 5*x(n-7)
sys3 = @(x_seq) ([zeros(1,1), x_seq(1:end-1)]).^2;  % System 3: y(n) = x^2(n-1)
```

```
%% Verify Time Invariance (TI)
% Shift the input signal by delay_k
x_delayed = [zeros(1, delay_k), x(1:end-delay_k)];

% System 1
Y_of_delayed_input1 = sys1(x_delayed);
Y_original1 = sys1(x);
Delayed_Y_output1 = [zeros(1, delay_k), Y_original1(1:end-delay_k)];
is_TI_1 = isequal(Y_of_delayed_input1, Delayed_Y_output1);

% System 2
Y_of_delayed_input2 = sys2(x_delayed);
Y_original2 = sys2(x);
Delayed_Y_output2 = [zeros(1, delay_k), Y_original2(1:end-delay_k)];
is_TI_2 = isequal(Y_of_delayed_input2, Delayed_Y_output2);
```

```
% System 3
Y_of_delayed_input3 = sys3(x_delayed);
Y_original3 = sys3(x);
Delayed_Y_output3 = [zeros(1, delay_k), Y_original3(1:end-delay_k)];
is_TI_3 = isequal(Y_of_delayed_input3, Delayed_Y_output3);
```

```
%% Print Results to Command Window
fprintf('\n=====================================================\n');
fprintf('TASKS 11 & 13: TIME INVARIANCE AND LTI VERIFICATION\n');
fprintf('=====================================================\n\n');

fprintf('System 1: y(n) = x(n-3)\n');
fprintf('  - Time Invariant Check: %d (1=True, 0=False)\n', is_TI_1);
fprintf('  - Linearity Check (Task 8): 1\n');
fprintf('  - LTI Status: YES (System is Linear and Time-Invariant)\n\n');

fprintf('System 2: y(n) = 5 * x(n-7)\n');
fprintf('  - Time Invariant Check: %d (1=True, 0=False)\n', is_TI_2);
fprintf('  - Linearity Check (Task 9): 1\n');
fprintf('  - LTI Status: YES (System is Linear and Time-Invariant)\n\n');

fprintf('System 3: y(n) = x^2(n-1)\n');
fprintf('  - Time Invariant Check: %d (1=True, 0=False)\n', is_TI_3);
fprintf('  - Linearity Check (Task 10): 0\n');
fprintf('  - LTI Status: NO (System is Time-Invariant, but Non-Linear)\n');
fprintf('=====================================================\n\n');
```

```
==================================================
TASKS 11 & 13: TIME INVARIANCE AND LTI VERIFICATION
==================================================

System 1: y(n) = x(n-3)
  - Time Invariant Check: 1 (1=True, 0=False)
  - Linearity Check (Task 8): 1
  - LTI Status: YES (System is Linear and Time-Invariant)

System 2: y(n) = 5 * x(n-7)
  - Time Invariant Check: 1 (1=True, 0=False)
  - Linearity Check (Task 9): 1
  - LTI Status: YES (System is Linear and Time-Invariant)

System 3: y(n) = x^2(n-1)
  - Time Invariant Check: 1 (1=True, 0=False)
  - Linearity Check (Task 10): 0
  - LTI Status: NO (System is Time-Invariant, but Non-Linear)
==================================================
```

14

```
% random_signal_lti_proof.m
rng(42); % Fixed seed for consistent visualization
N_samples = 40;
n = 1:N_samples;

% Random Inputs
x1 = randn(1, N_samples);
x2 = randn(1, N_samples);
c1 = 1.5; c2 = -0.8;
delay_k = 5;

% Custom LTI System: Moving Average Filter y(n) = 0.5x(n) + 0.5x(n-1)
system_func = @(x) filter([0.5 0.5], 1, x);

figure('Name', 'Random Signal LTI Verification', 'Position', [100 100 800 600]);
```

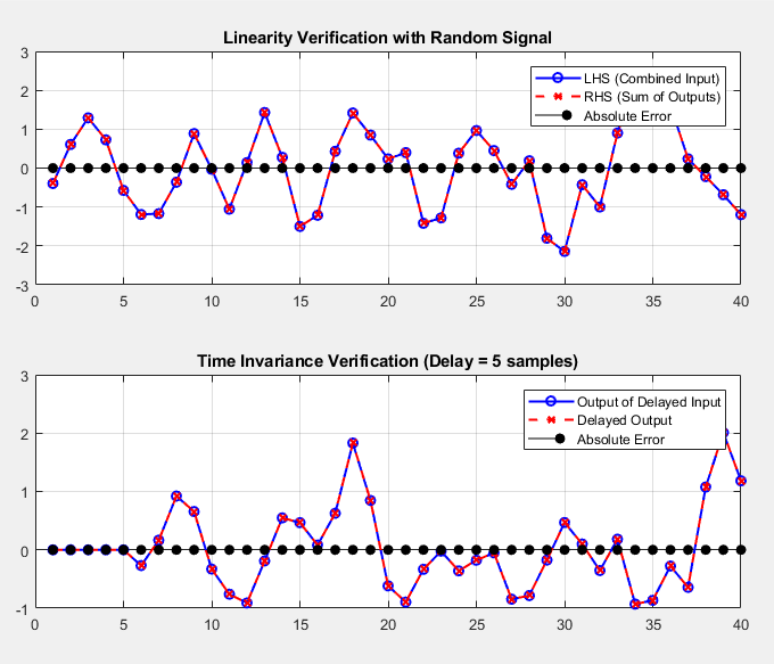
```
%% Linearity Verification
LHS_Lin = system_func(c1*x1 + c2*x2);
RHS_Lin = c1*system_func(x1) + c2*system_func(x2);
error_Lin = abs(LHS_Lin - RHS_Lin);

subplot(2,1,1);
plot(n, LHS_Lin, 'b-o', 'LineWidth', 1.5); hold on;
plot(n, RHS_Lin, 'r--x', 'LineWidth', 1.5);
stem(n, error_Lin, 'k', 'filled'); % Plots the error (should remain at 0)
title('Linearity Verification with Random Signal');
legend('LHS (Combined Input)', 'RHS (Sum of Outputs)', 'Absolute Error');
grid on; hold off;
```

```
%% Time Invariance Verification
x1_delayed = [zeros(1, delay_k), x1(1:end-delay_k)];
Y_of_delayed = system_func(x1_delayed);

Y_original = system_func(x1);
Delayed_Y = [zeros(1, delay_k), Y_original(1:end-delay_k)];
error_TI = abs(Y_of_delayed - Delayed_Y);

subplot(2,1,2);
plot(n, Y_of_delayed, 'b-o', 'LineWidth', 1.5); hold on;
plot(n, Delayed_Y, 'r--x', 'LineWidth', 1.5);
stem(n, error_TI, 'k', 'filled'); % Plots the error (should remain at 0)
title(['Time Invariance Verification (Delay = ', num2str(delay_k), ' samples)']);
legend('Output of Delayed Input', 'Delayed Output', 'Absolute Error');
grid on; hold off;
```


Linearity Verification with Random Signal
LHS (Combined Input)
RHS (Sum of Outputs)
Absolute Error
3
2
1
0
-1
-2
-3
0
5
10
15
20
25
30
35
40

Time Invariance Verification (Delay = 5 samples)
Output of Delayed Input
Delayed Output
Absolute Error
3
2
1
0
-1
0
5
10
15
20
25
30
35
40